Sesión 1: configuración github

Guía de GitHub

GitHub es una plataforma en línea que combina control de versiones con colaboración en equipo, perm
software de manera segura. Su base está en **Git**, una herramienta que registra los cambios en los archi
Aprender a manejar GitHub es esencial porque facilita el trabajo en equipo, el seguimiento de la evoluci
personas editan el mismo archivo. Además, es una vitrina profesional: muchos empleadores revisan pe

Es momento de poner en práctica la guía y dar tus primeros pasos en GitHub. Empieza creando tu cuen
tu nombre y correo. Luego, sigue los comandos paso a paso para crear un repositorio local, añadir un a
repositorio con GitHub y sube los cambios para verlos reflejados en línea. Al completar estos pasos, hab
listo para trabajar con proyectos y colaborar con otros.

---

1) Crear una cuenta en GitHub

Pasos sencillos:

1. Abre el navegador y ve a **https://github.com**.
2. Haz clic en **Sign up** (Registrarse).
3. Completa:
    - **Username** (nombre de usuario): el identificador público (ej.: `tu-usuario`).
    - **Email**: tu correo electrónico.
    - **Password**: contraseña segura.
4. Verifica tu correo cuando GitHub te envíe el email.
5. Elige el plan (el **Free** es suficiente para comenzar).
6. Opcional pero recomendable: habilita **2FA (autenticación en dos pasos)** en Settings → Security pc

Consejo: anota tu nombre de usuario y el email exacto que usaste: los necesitarás para configuracione

---

2) Configuración inicial en tu ordenador (cor

Para comenzar a trabajar con Git y GitHub en tu computadora con Windows, el primer paso es instalar G en una terminal sencilla y práctica. Para hacerlo, visita la página oficial de git-scm.com ↗, descarga la opciones por defecto. Git Bash es importante porque te brinda un entorno similar al de Linux, lo que facil uniforme y adquirir habilidades que son estándar en el mundo del desarrollo de software.

Antes de usar Git en tu PC, configura tu nombre y correo (asociados a tus commits):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-email@example.com"
```

Qué hace cada línea

- `git config --global user.name "Tu Nombre"` : guarda tu nombre globalmente en Git (aparecerá en lo
- `git config --global user.email "tu-email@example.com"` : guarda tu e-mail.

Cómo verificar

```
git config --list
```

Verás `user.name=Tu Nombre` y `user.email=tu-email@example.com` .

---

3) Crear tu primer repositorio — dos manera

Opción A — Crear en GitHub y clonar (recomendado cuando empiezas)

1. En GitHub, haz clic en **New repository**.
2. Pon un **Repository name**: por ejemplo `mi-primer-repo` .
3. Opcional: descripción. Elige **Public** o **Private**.
4. (Opcional) Marca *Initialize this repository with a README* — si lo haces, **clónalo** en lugar de intentar

Luego en tu terminal:

```
git clone https://github.com/tu-usuario/mi-primer-repo.git
cd mi-primer-repo
```

Qué hace cada línea

- `git clone URL` : descarga una copia del repositorio remoto a tu ordenador.
- `cd mi-primer-repo` : entra en la carpeta del proyecto.

Comprobar

- `ls` (o `dir`) mostrará archivos (README si lo inicializaste).

- `git remote -v` mostrará `origin` apuntando al URL.

Opción B — Crear localmente y subirlo a GitHub

Útil si ya tienes archivos en tu PC y quieres subirlos a GitHub.

1. Crea carpeta local y agrega archivos:

```
mkdir mi-primer-repo
cd mi-primer-repo
echo "# Mi Primer Repo" > README.md
```

2. Inicializa Git (ver más abajo `git init`):

```
git init
```

3. Añade archivos al área de preparación (staging) y haz el primer commit:

```
git add README.md
git commit -m "Primer commit: añadir README"
```

4. En GitHub crea un **nuevo repositorio** con el mismo nombre `mi-primer-repo` **sin** marcar "Initialize wi
   muestra (ej.: `https://github.com/tu-usuario/mi-primer-repo.git`).
5. Vincula tu repositorio local con el remoto y sube:

```
git branch -M main
git remote add origin https://github.com/tu-usuario/mi-primer-repo.git
git push -u origin main
```

Qué hace cada línea

- `git branch -M main` : renombra la rama actual a `main` (convención moderna).
- `git remote add origin URL` : añade el *remote* llamado `origin` que apunta a GitHub.
- `git push -u origin main` : sube la rama `main` al remoto y establece `origin/main` como su rama re

Cómo verificar

- En GitHub: refresca la página del repositorio: verás tu README y commit.
- Local: `git remote -v` y `git status` (debería decir "on branch main" y "nothing to commit, working

Nota sobre autenticación: cuando haces `git push` por HTTPS, Git puede pedirte usuario y "contraseña". operaciones Git por HTTPS — normalmente pide un **Personal Access Token (PAT)** o usar **SSH**. Si te pide c generar un PAT o configurar una clave SSH.

---

4) Comandos básicos de Git

Cuando trabajes con Git, es fundamental conocer los **comandos básicos**, ya que en muchas situacione crear un repositorio, registrar cambios o subir tu trabajo a GitHub. Estos comandos son la base del conti proyectos personales hasta entornos profesionales. Por ejemplo, en **MLOps** se usan constantemente pa configuraciones de modelos, asegurando que cada cambio quede documentado y que el equipo pued permitirá avanzar con confianza en cualquier proyecto.

`git init`

Qué hace: convierte una carpeta en un repositorio Git local (crea la carpeta oculta `.git` que contiene

Ejemplo

```
mkdir proyecto-demo
cd proyecto-demo
git init
```

Qué hace cada línea

- `mkdir proyecto-demo` : crea una carpeta.
- `cd proyecto-demo` : entra en la carpeta.
- `git init` : inicializa Git en esa carpeta.

Cómo comprobar

- `ls -a` mostrará `.git` .
- `git status` mostrará "On branch main" o indicará que aún no hay commits.

`git add`

Analogía: el *staging area* (área de preparación) es como el carrito de compras: eliges los archivos que

Qué hace: mueve cambios (archivos nuevos o modificados) al **staging area** para que estén listos para

Ejemplos

- Añadir un archivo específico:

  ```
  git add archivo.txt
  ```

- Añadir todos los cambios (incluyendo archivos nuevos y eliminados):

  ```
  git add -A
  ```

- Añadir todos los archivos del directorio actual:

```
git add .
```

Qué hace cada línea

- `git add archivo.txt` : marca `archivo.txt` para el próximo commit.
- `git add -A` : prepara **todos** los cambios (nuevos, modificados y eliminados).
- `git add .` : prepara los cambios en la carpeta actual (no siempre idéntico a `-A` en versiones ar

Cómo comprobar

```
git status
```

- Verás una sección "Changes to be committed" con los archivos en color (estarán "staged").
- Si no aparecen, el archivo no está en el staging area.

`git commit`

Analogía: hacer un *checkout* en una máquina del tiempo: guarda una foto del proyecto en ese momen

Qué hace: crea un nuevo commit con los archivos que están en el staging area.

Ejemplo simple

```
git commit -m "Añadir archivo de ejemplo y README"
```

Qué hace cada parte

- `git commit` : crea el commit.
- `-m "mensaje"` : mensaje corto describiendo el cambio.

Consejo de mensajes: escribe en presente imperativo y breve (ej.: `Corrige bug en login` , `Añade README in` `-m` varias veces.

Cómo comprobar

- `git log --oneline` muestra lista de commits (cada línea: ID corto y mensaje).
- `git show HEAD` muestra el último commit y los cambios incluidos.
- `git status` debería decir "nothing to commit, working tree clean".

`git push`

Qué hace: sube los commits desde tu repositorio local al repositorio remoto (ej. GitHub).

Ejemplo (empujar rama `main` a `origin`):

```
git push -u origin main
```

Qué hace cada parte

- `git push` : comando para subir commits.
- `-u` (o `--set-upstream`): establece la rama remota por defecto para `main` (así la próxima vez sól
- `origin` : nombre del remoto (por convención el remoto creado con `git remote add` se llama `ori`
- `main` : rama que quieres subir.

Cómo comprobar

- `git status` — si está sincronizado verás "Your branch is up to date with 'origin/main'."
- `git log origin/main..main` — si no muestra nada, significa que no hay commits locales no empujc
- Mirar en GitHub: tu commit debe aparecer en la interfaz web.

Errores comunes

- `failed to push some refs` — puede significar que la rama remota tiene commits que tú no tienes:
- `authentication failed` — revisa PAT/SSH como expliqué antes.

`git clone`

Qué hace: clona (copia) un repositorio remoto a tu máquina local.

Ejemplo

```
git clone https://github.com/tu-usuario/mi-primer-repo.git
```

Qué hace cada parte

- `git clone URL` : copia el repo remoto y crea un folder con el nombre del proyecto.

Cómo comprobar

- Aparecerá una carpeta `mi-primer-repo` .
- Entra `cd mi-primer-repo` y ejecuta `git remote -v` para ver `origin` apuntando al URL.

---

5) Flujo de trabajo básico

Aquí tienes un flujo simple y claro que funciona para la mayoría de los proyectos pequeños.

1. **Clonar** el repositorio (una sola vez):

```
git clone https://github.com/tu-usuario/mi-primer-repo.git
cd mi-primer-repo
```

2. **Crear una rama** (opcional pero recomendable para trabajar en cambios sin tocar `main`):

```
git checkout -b mi-cambio
```

   - `checkout -b` crea y cambia a la rama `mi-cambio`.
3. **Editar archivos** con tu editor favorito.
4. **Preparar los cambios** (staging):

```
git add archivo1.txt archivo2.txt
# o git add .
```

5. **Hacer commit** con mensaje claro:

```
git commit -m "Descripción corta: Qué y por qué"
```

6. **Subir tu rama** al remoto:

```
git push -u origin mi-cambio
```

7. **Abrir un Pull Request (PR)** en GitHub: compara `mi-cambio` con `main` y solicita revisión / merge.
8. Tras el merge en GitHub, vuelve a la rama `main` y actualiza:

```
git checkout main
git pull origin main
```

Por qué usar ramas: te permiten trabajar sin romper la rama principal; luego pedir que alguien revise tu

---

6) Comandos útiles extra y comprobaciones

- `git status` — muestra qué archivos están sin seguimiento, en staging o modificados.
- `git log --oneline --graph --all` — historial resumido (útil para ver la historia).
- `git remote -v` — muestra los remotos (ej. `origin`).
- `git branch` — lista ramas locales; `git branch -a` incluye remotas.
- `git diff` — ver cambios no staged; `git diff --staged` ver cambios staged.
- `git show HEAD` — ver último commit y su diff.

---

7) Archivos que no quieres subir: `.gitignore`

Crea un archivo `.gitignore` para decirle a Git qué NO incluir (ej.: archivos temporales, credenciales, `noc`

Ejemplo básico:

```
# .gitignore
node_modules/
*.log
.env
```

Agregar y commitear:

```
git add .gitignore
git commit -m "Añadir .gitignore"
```

8) Ejemplo: desde cero hasta GitHub

Sustituye `tu-usuario` por tu usuario real y `mi-primer-repo` por el nombre que quieras.

```
# 1. Crear carpeta local
mkdir mi-primer-repo
cd mi-primer-repo

# 2. Inicializar git
git init                 # crea .git

# 3. Crear un README
echo "# Mi Primer Repo" > README.md

# 4. Preparar y commitear
git add README.md
git commit -m "Primer commit: README"

# 5. Preparar rama main
git branch -M main

# 6. Conectar con GitHub (crea primero un repo vacío en GitHub)
git remote add origin https://github.com/tu-usuario/mi-primer-repo.git

# 7. Subir a GitHub
git push -u origin main
```

Verificación en cada paso

- Tras `git init` : `ls -a` → `.git` existe.
- Tras `git add` : `git status` → archivo en *staged*.
- Tras `git commit` : `git log --oneline` → verás tu commit.
- Tras `git push` : `git status` → "up to date with origin/main" y en GitHub verás el archivo.

---

9) Buenas prácticas

- Haz commits pequeños y frecuentes (cada cambio lógico).
- Escribe mensajes claros: breve título (máx ~50 caracteres) + detalle si hace falta.
- No subas secretos (contraseñas, claves). Guarda esos archivos fuera o usa variables de entorno
- Usa ramas para nuevas funcionalidades o arreglos.
- Antes de `push` , siempre `git pull` para traer cambios remotos y evitar conflictos.

---

10) Problemas comunes y frases que verás

- `nothing to commit, working tree clean` → no hay cambios que commitear.
- `Untracked files:` → archivos nuevos no añadidos (usa `git add`).
- `Changes not staged for commit:` → modificaste archivos pero no los agregaste al staging.
- `error: failed to push some refs` → tu remoto tiene cambios que tú no tienes; usa `git pull` y resu
- `Authentication failed` → revisa si necesitas usar **Personal Access Token (PAT)** o configurar **SSH k**

Cheat sheet rápido

- `git init` — inicializar repo local.
- `git clone URL` — clonar repo remoto.
- `git status` — ver estado actual.
- `git add <archivo>` / `git add .` / `git add -A` — mover al staging.
- `git commit -m "mensaje"` — crear commit.
- `git push origin <rama>` — subir commits.
- `git pull` — traer cambios y fusionar.
- `git checkout -b <rama>` — crear y cambiar a nueva rama.
- `git branch` — listar ramas.
- `git remote -v` — ver remotos.

Última modificación: jueves, 25 de septiembre de 2025, 18:45